Práctica de Redis — Introducción a Redis

Integrantes: Matias Benavides Sandoval — 2025102376
Curso: Bases de Datos II (IC-4302)

---

1. Strings

8. Añade el texto `" (IC-4302)"` al final del valor de `nombre_curso`.

Comando de Redis:

```
APPEND nombre_curso " (IC-4302)"
```

Salida o resultado:

```
(integer) 27
```

---

9. Obtén el nuevo valor de `nombre_curso`.

Comando de Redis:

```
GET nombre_curso
```

Salida o resultado:

```
"Bases de Datos II (IC-4302)"
```

---

10. Obtén la longitud del valor almacenado en `nombre_curso`.

Comando de Redis:

```
STRLEN nombre_curso
```

Salida o resultado:

```
(integer) 27
```

---

11. Establece múltiples claves a la vez: `profesor` con valor `"Kenneth Obando"` y `semestre` con valor `"I"`.

Comando de Redis:

```
MSET profesor "Kenneth Obando" semestre "II"
```

Salida o resultado:

```
OK
```

---

12. Obtén los valores de `profesor` y `semestre` en un solo comando.

Comando de Redis:

```
MGET profesor semestre
```

Salida o resultado:

```
1) "Kenneth Obando"
2) "II"
```

---

2. Listas — `motores_db`

5. Obtén los elementos del índice 1 al 2.

Comando de Redis:

```
LRANGE motores_db 1 2
```

Salida o resultado:

```
1) "PostgeSQL"
2) "Oracle"
```

---

6. Obtén y elimina el primer elemento (izquierda) de la lista `motores_db`. ¿Qué elemento se eliminó?

Comando de Redis:

```
LPOP motores_db
```

Salida o resultado:

```
"SQL Server"
```

> **Elemento eliminado:** `SQL Server`.

---

7. Obtén y elimina el último elemento (derecha) de la lista `motores_db`. ¿Qué elemento se eliminó?

Comando de Redis:

```
RPOP motores_db
```

Salida o resultado:

```
"MySQL"
```

> **Elemento eliminado:** `MySQL`.

---

8. Obtén la longitud actual de la lista `motores_db`.

Comando de Redis:

```
LLEN motores_db
```

Salida o resultado:

```
(integer) 2
```

---

9. Obtén los elementos restantes de la lista `motores_db`.

Comando de Redis:

```
LRANGE motores_db 0 -1
```

Salida o resultado:

```
1) "PostgeSQL"
2) "Oracle"
```

---

3. Sets — `temas_curso`

5. Verifica si `"Seguridad"` es miembro del set `temas_curso`.

Comando de Redis:

```
SISMEMBER temas_curso "Seguridad"
```

Salida o resultado:

```
(integer) 1
```

---

6. Verifica si `"Big Data"` es miembro del set `temas_curso`.

Comando de Redis:

```
SISMEMBER temas_curso "Big Data"
```

Salida o resultado:

```
(integer) 0
```

---

7. Elimina `"Transacciones"` del set `temas_curso`. (es que lo escribi mal profe)

Comando de Redis:

```
SREM temas_curso "Transacciones"
```

Salida o resultado:

```
(integer) 0
```

Verificación y corrección realizada:

```
SMEMBERS temas_curso
```

```
1) "Transcciones"
2) "Replicacion"
3) "Seguridad"
4) "Optimizacion"
```

```
SREM temas_curso "Transcciones"
```

```
(integer) 1
```

```
SMEMBERS temas_curso
```

```
1) "Replicacion"
2) "Seguridad"
3) "Optimizacion"
```

---

8. Obtén la cantidad de elementos (cardinalidad) en el set `temas_curso`.

Comando de Redis:

```
SCARD temas_curso
```

Salida o resultado:

```
(integer) 4
```

Valor final después de la corrección:

```
SCARD temas_curso
```

```
(integer) 3
```

---

9. Obtén todos los miembros restantes del set `temas_curso`.

Comando de Redis:

```
SMEMBERS temas_curso
```

Salida o resultado:

```
1) "Replicacion"
2) "Seguridad"
3) "Optimizacion"
```

---

4. Hashes — `estudiante:101`

5. Obtén solo los nombres de los campos (keys) del hash `estudiante:101`.

Comando de Redis:

```
HKEYS estudiante:101
```

Salida o resultado:

```
1) "nombre"
2) "apellido"
3) "email"
```

---

6. Obtén solo los valores de los campos del hash `estudiante:101`.

Comando de Redis:

```
HVALS estudiante:101
```

Salida o resultado:

```
1) "Ana"
2) "Solano"
3) "ana.solano@email.com"
```

---

7. Elimina el campo `carnet` del hash `estudiante:101`.

Comando de Redis:

```
HDEL estudiante:101 carnet
```

Salida o resultado:

```
(integer) 1
```

---

8. Obtén el número de campos en el hash `estudiante:101`.

Comando de Redis:

```
HLEN estudiante:101
```

Salida o resultado:

```
(integer) 3
```

---

9. Obtén todos los campos y valores restantes de `estudiante:101`.

Comando de Redis:

```
HGETALL estudiante:101
```

Salida o resultado:

```
1) "nombre"
2) "Ana"
3) "apellido"
4) "Solano"
5) "email"
6) "ana.solano@email.com"
```

5. Sorted Sets — `ranking_proyectos`

6. Obtén el ranking (posición, basada en 0) de `"Proyecto Gama"` (ordenado de menor a mayor score).

Comando de Redis:

```
ZRANK ranking_proyectos "Proyecto Gama"
```

Salida o resultado:

```
(integer) 2
```

Intento previo con clave en singular:

```
ZRANK ranking_proyecto "Proyecto Gama"
```

```
(nil)
```

---

7. Obtén el score de `"Proyecto Alfa"`.

Comando de Redis:

```
ZSCORE ranking_proyectos "Proyecto Alfa"
```

Salida o resultado:

```
"95"
```

---

8. Obtén la cantidad de miembros en `ranking_proyectos`.

Comando de Redis:

```
ZCARD ranking_proyectos
```

Salida o resultado:

```
(integer) 4
```

Verificación adicional — orden de mayor a menor con scores:

```
ZREVRANGE ranking_proyectos 0 -1 WITHSCORES
```

```
1) "Proyecto Alfa"
2) "95"
3) "Proyecto Gama"
4) "92"
5) "Proyecto Delta"
6) "88"
```

```
7) "Proyecto Beta"
8) "88"
```

6. Administración de Claves

8. Establece un tiempo de expiración de 60 segundos para la clave `temas_curso`.

Comando de Redis:

```
EXPIRE temas_curso 60
```

Salida o resultado:

```
(integer) 1
```

---

9. Consulta el tiempo restante (en segundos) antes de que la clave `temas_curso` expire. Ejecútalo varias veces para ver cómo disminuye.

Comando de Redis (ejecutado varias veces):

```
TTL temas_curso
```

Salida o resultado (disminuye con cada ejecución):

```
(integer) 53
(integer) 52
(integer) 51
(integer) 50
(integer) 49
(integer) 48
...
(integer) 22
...
(integer) 3
(integer) 1
(integer) -2
```

La clave expiró al llegar a `-2` (no existe).

---

10. Consulta el tiempo de expiración de una clave que no tiene expiración, como `nombre_curso`. ¿Qué devuelve?

Comando de Redis:

```
TTL nombre_curso
```

Salida o resultado:

```
(integer) -1
```

Devuelve `-1`: la clave existe pero no tiene expiración.

---

11. Consulta el tiempo de expiración de una clave que no existe, como `creditos`. ¿Qué devuelve?

Comando de Redis:

```
TTL creditos
```

Salida o resultado:

```
(integer) -2
```

Devuelve `-2`: la clave no existe.

---

12. Elimina todas las claves de la base de datos actual.

Comando de Redis:

```
FLUSHDB
```

Salida o resultado:

```
OK
```

---

13. Verifica que no queden claves.

Comando de Redis:

```
KEYS *
```

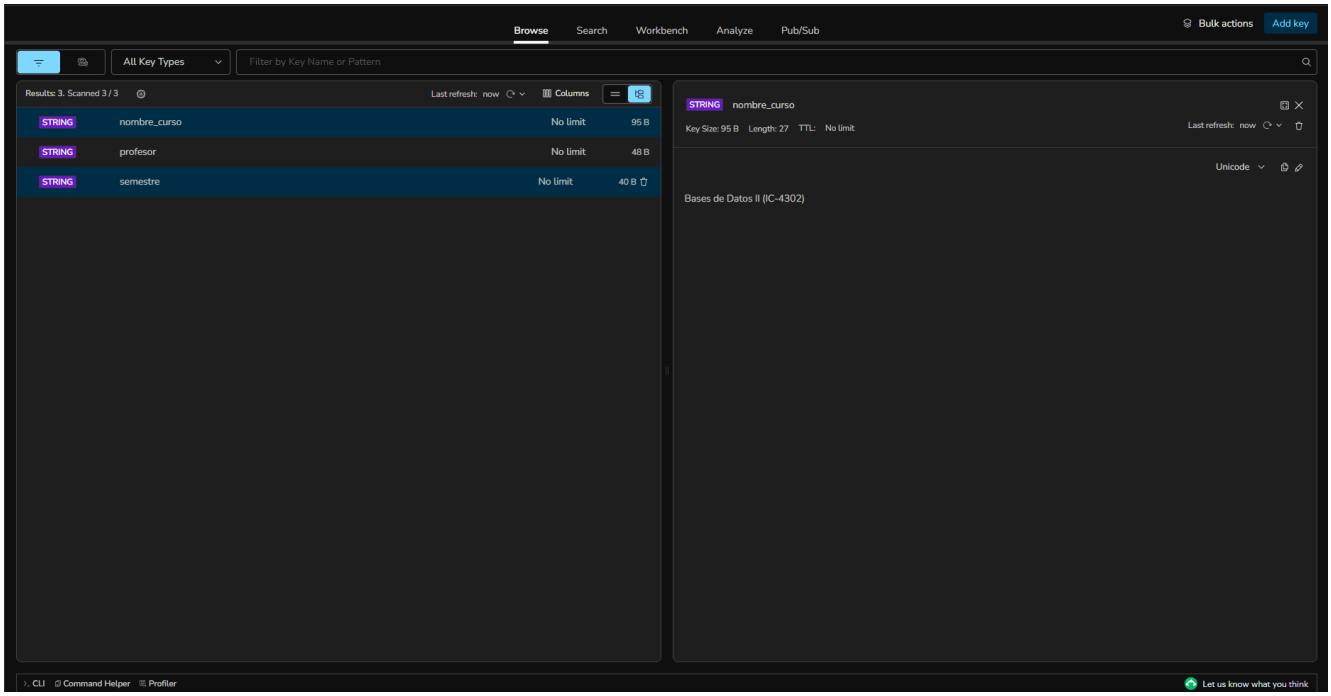
Salida o resultado:

```
(empty array)
```

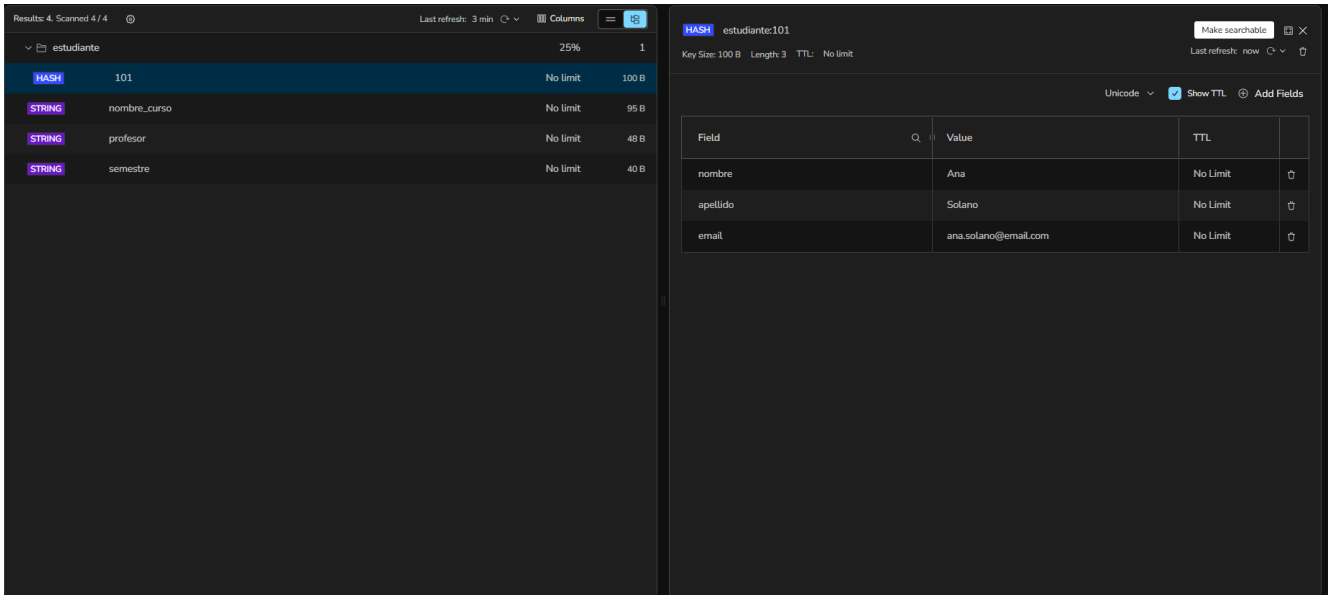
---

7. Capturas de pantalla — RedisInsight

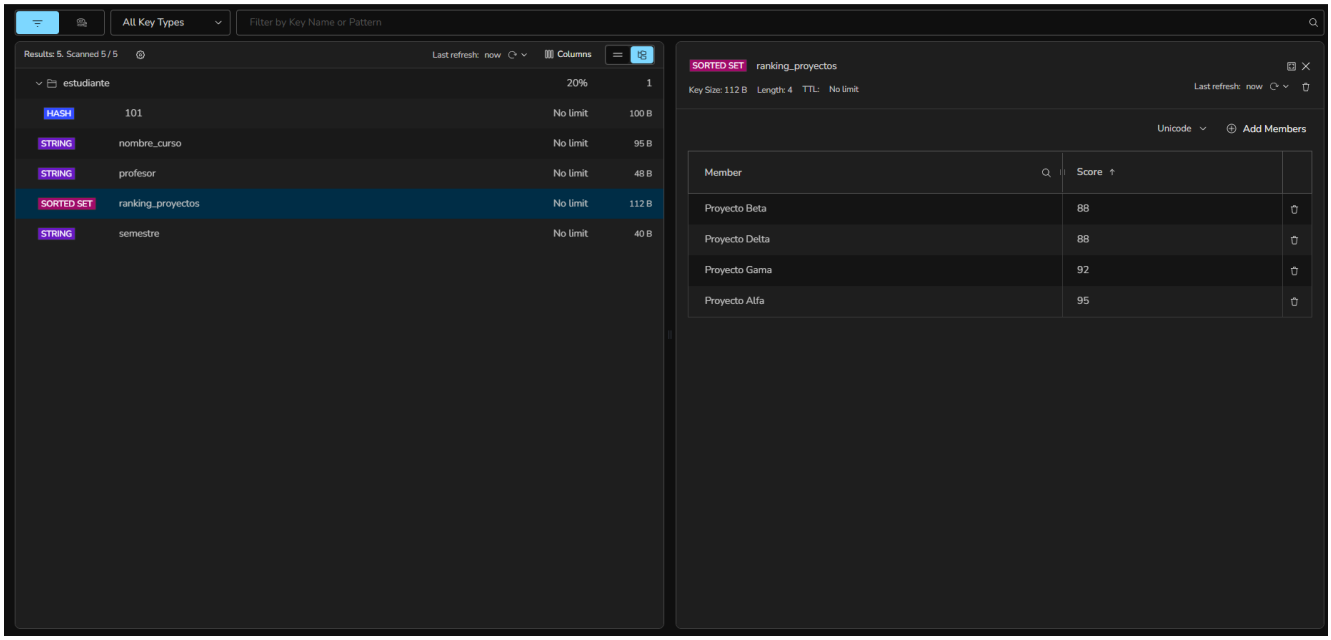
Captura 1 — Strings (`nombre_curso`)



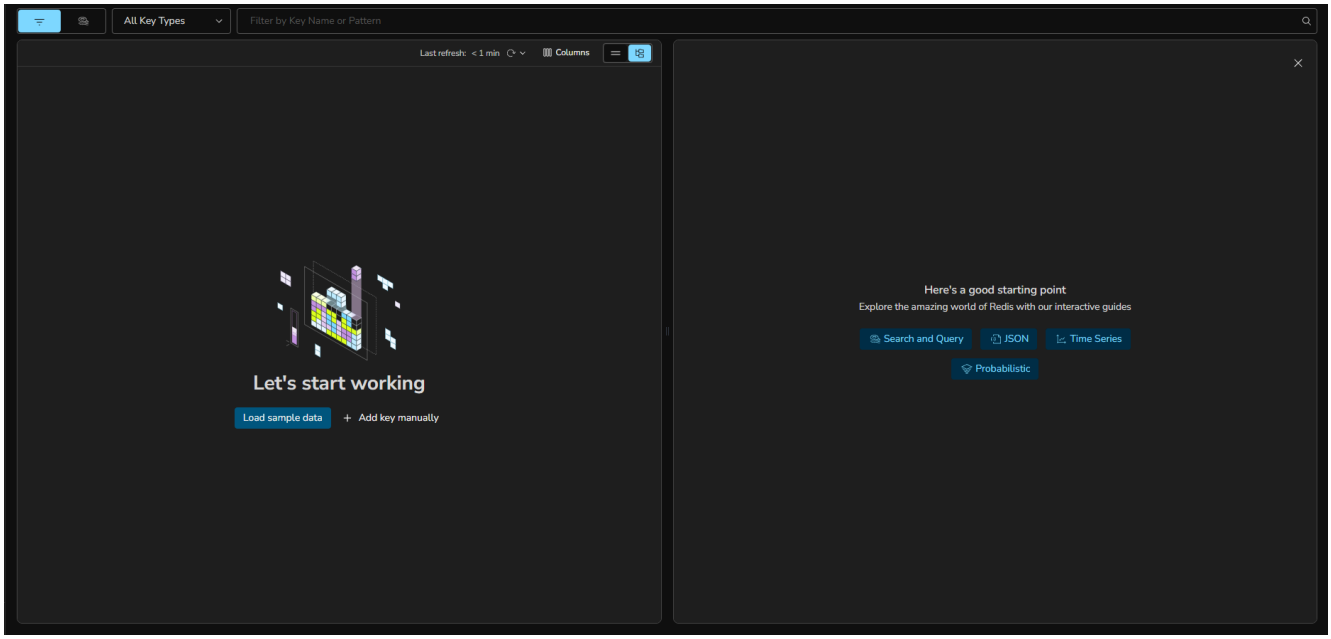
Captura 2 — Hash (`estudiante:101`)



Captura 3 — Sorted Set (`ranking_proyectos`)



Captura 4 — Base vacía tras `FLUSHDB`


All Key Types
Filter by Key Name or Pattern
Last refresh: < 1 min
Columns
Let's start working
Load sample data
Add key manually
Here's a good starting point
Explore the amazing world of Redis with our interactive guides
Search and Query
JSON
Time Series
Probabilistic